

Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний
інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра ІІІ

Звіт

з лабораторної роботи №5 з дисципліни
«Системне програмне забезпечення»

„Мкрозасоби мови асемблер”

Виконав ІІ-45 Янов Б. Є.

Перевірив Бабич Б. Б.

Київ 2026

Лабораторна робота №5

Мкррозасоби мови асемблер

Завдання:

- 1) переписати програму комп'ютерного практикуму No 2 з використанням одного макроса;
- 2) переписати програму комп'ютерного практикуму No 3 з використанням макросів та передачею параметрів в них;
- 3) переписати одну програму (на вибір студента) комп'ютерного практикуму No 4 з використанням макросів та залученням міток в тілі макросу.

Текст програми 2

```
.model small
.stack 100h

print_str macro str
        lea dx, str
        mov ah, 09h
        int 21h
endm

.data
    msg_input db 'Enter an integer: $'
    msg_res   db 13, 10, 'Result (-34): $'
    msg_err   db 13, 10, 'Error: Invalid input! (Numbers only)$'
    msg_ovf   db 13, 10, 'Error: Overflow/Underflow! Keep it 16-
bit.$'

    ; Buf for 10th DOS function: max len, real len, char array
    buffer    db 7, 0, 7 dup(0)
    num       dw 0
    is_neg    db 0

.code
main proc
        mov     ax, @data
        mov     ds, ax

        print_str msg_input

        call    InputProc
        jc      exit_prog          ; Carry Flag = 1 =>
error

        ; Var. 19: -34
        mov     ax, num
        sub     ax, 34
        jo      overflow_err
```

```

        push        ax                ; save ax
        print_str msg_res
        pop         ax                ; restore ax

        call        OutputProc
        jmp         exit_prog

overflow_err:
        print_str msg_ovf
        jmp         exit_prog

exit_prog:
        mov         ax, 4c00h
        int         21h
main endp

InputProc proc

        lea         dx, buffer
        mov         ah, 0Ah
        int         21h

        lea         si, buffer+2      ; ptr of symbols
        mov         cl, buffer+1      ; num of symbols
        xor         ch, ch
        cmp         cx, 0
        je          input_error       ; zero characters

        xor         bx, bx            ; num accumulator
        mov         is_neg, 0

        ; check for sign
        mov         al, [si]
        cmp         al, '-'
        jne         check_plus
        mov         is_neg, 1
        inc         si
        dec         cx
        jmp         parse_loop

check_plus:
        cmp         al, '+'
        jne         parse_loop
        inc         si
        dec         cx

parse_loop:
        cmp         cx, 0

```

```

je            parse_done

mov          al, [si]
; check if it is a number (0-9)
cmp          al, '0'
jl           input_error
cmp          al, '9'
jg           input_error

sub          al, '0'           ; char -> num
xor          ah, ah
push        cx                ; save counter

mov          cx, ax            ; cx = current num
mov          ax, bx
mov          dx, 10
imul         dx                ; ax = ax * 10
jo           ovf_error

add          ax, cx            ; ax = ax + current num
jo           ovf_error

mov          bx, ax
pop          cx                ; restore counter

inc          si
dec          cx
jmp          parse_loop

parse_done:
cmp          is_neg, 1
jne          save_number
neg          bx                ; swap left bit -> num
is positive
jo           ovf_error

save_number:
mov          num, bx
clc                                ; Carry Flag = 0
ret

ovf_error:
pop          cx                ; clear stack when
exiting loop

print_str msg_ovf
stc                                ; Carry Flag = 1
ret

input_error:
print_str msg_err

```

```

                                stc
                                ret

InputProc endp

OutputProc proc
                                cmp     ax, 0
                                jge     start_convert    ; skip - if ax >= 0

                                ; print -
                                push     ax
                                mov      ah, 02h
                                mov      dl, '-'
                                int       21h
                                pop      ax
                                neg      ax              ; swap left bit

                                start_convert:
                                xor      cx, cx          ; reset cx
                                mov      bx, 10

                                divide_loop:
                                xor      dx, dx
                                div       bx             ; ax = ax / 10, dx =
rem                                push    dx            ; push to stack
                                inc      cx
                                test     ax, ax
                                jnz      divide_loop

                                print_loop:
                                pop      dx
                                add      dl, '0'        ; convert num to char
                                mov      ah, 02h
                                int       21h
                                loop     print_loop

                                ret

OutputProc endp

end main

```

Текст програми 3

```

.model small
JUMPS
.stack 100h

```

```

print_str macro str
    lea dx, str
    mov ah, 09h
    int 21h

```

endm

```
input_num macro var, str
    print_str str
    call     InputProc
    jc       exit_prog      ; Exit on error
    mov      var, ax
```

endm

```
input_str macro buffer
    lea dx, buffer
    mov ah, 0Ah
    int 21h
```

endm

.data

; Input messages

msg_a db 'Enter a: \$'

msg_b db 13, 10, 'Enter b: \$'

msg_x db 13, 10, 'Enter x: \$'

; Output messages

msg_res db 13, 10, 'Result Z = \$'

msg_rem db ' (rem: \$'

msg_rem_end db ')\$'

; Error messages

msg_err db 13, 10, 'Error: Invalid input! (Numbers only)\$'

msg_ovf db 13, 10, 'Error: Overflow/Underflow! Limit to 16-bit.\$'

; Variables

buffer db 7, 0, 7 dup(0)

val_a dw 0

val_b dw 0

val_x dw 0

is_neg db 0

res_rem dw 0

.code

main proc

mov ax, @data

mov ds, ax

input_num val_a, msg_a

input_num val_b, msg_b

input_num val_x, msg_x

mov ax, val_x

cmp ax, 0

jg calc_greater ; x > 0

```

je      calc_equal      ; x == 0
jl      calc_less       ; x < 0

```

; x > 0 => ax^2 + b/x

calc_greater:

```

; Calculate ax^2
mov     ax, val_x
imul    ax               ; ax = x^2 (dx:ax)
jo      overflow_err
imul    val_a            ; ax = a * x^2
jo      overflow_err
mov     bx, ax           ; bx = ax^2

; Calculate b/x
mov     ax, val_b
cwd                     ; Extend ax to dx:ax for idiv
idiv    val_x            ; ax, dx = rem

; Calculate (ax^2) + (b/x)
add     ax, bx
jo      overflow_err

; Save remaining
mov     res_rem, dx
jmp     print_res

```

; x = 0 => a + 2b

calc_equal:

```

mov     ax, val_b
mov     cx, 2
imul    cx               ; ax = 2b
jo      overflow_err

add     ax, val_a        ; ax = a + 2b
jo      overflow_err

jmp     print_res

```

; x < 0 => ax^2 - bx

calc_less:

```

; Calculate ax^2
mov     ax, val_x
imul    ax
jo      overflow_err
imul    val_a
jo      overflow_err
mov     bx, ax           ; bx = ax^2

; Calculate bx
mov     ax, val_b
imul    val_x

```

```

        jo      overflow_err      ; ax = bx

        ; Calculate ax^2 - bx
        sub     bx, ax
        jo      overflow_err

        mov     ax, bx
        jmp     print_res

print_res:
        push    ax                ; Save result Z
        print_str msg_res
        pop     ax
        call    OutputProc        ; Print first part

        cmp     res_rem, 0        ; Skip if 0
        je      exit_prog

        ; Print rem
        print_str msg_rem

        mov     ax, res_rem
        call    OutputProc

        print_str msg_rem_end
        jmp     exit_prog

overflow_err:
        print_str msg_ovf
        jmp     exit_prog

exit_prog:
        mov     ax, 4c00h
        int     21h

main endp

```

```

; Copied from lab2
; Returns in AX

```

```

InputProc proc
        input_str buffer

        lea     si, buffer+2
        mov     cl, buffer+1
        xor     ch, ch
        cmp     cx, 0
        je      input_error

```

```

        xor        bx, bx
        mov        is_neg, 0

        mov        al, [si]
        cmp        al, '-'
        jne        check_plus
        mov        is_neg, 1
        inc        si
        dec        cx
        jmp        parse_loop

check_plus:
        cmp        al, '+'
        jne        parse_loop
        inc        si
        dec        cx

parse_loop:
        cmp        cx, 0
        je         parse_done

        mov        al, [si]
        cmp        al, '0'
        jl         input_error
        cmp        al, '9'
        jg         input_error

        sub        al, '0'
        xor        ah, ah
        push       cx

        mov        cx, ax
        mov        ax, bx
        mov        dx, 10
        imul       dx
        jo         ovf_error

        add        ax, cx
        jo         ovf_error

        mov        bx, ax
        pop        cx

        inc        si
        dec        cx
        jmp        parse_loop

parse_done:
        cmp        is_neg, 1
        jne        save_number

```

```

        neg     bx
        jo      ovf_error

save_number:
        mov     ax, bx           ; Return in AX
        cld
        ret

ovf_error:
        pop     cx
        print_str msg_ovf
        stc
        ret

input_error:
        print_str msg_err
        stc
        ret

InputProc endp

OutputProc proc
        cmp     ax, 0
        jge     start_convert

        push    ax
        mov     ah, 02h
        mov     dl, '-'
        int     21h
        pop     ax
        neg     ax

start_convert:
        xor     cx, cx
        mov     bx, 10

divide_loop:
        xor     dx, dx
        div     bx
        push    dx
        inc     cx
        test    ax, ax
        jnz     divide_loop

print_loop:
        pop     dx
        add     dl, '0'
        mov     ah, 02h
        int     21h
        loop    print_loop

        ret

OutputProc endp

```

end main

Текст програми 4

.model small

JUMPS

.stack 100h

print_str macro str

lea dx, str

mov ah, 09h

int 21h

endm

print_nl macro

mov ah, 02h

mov dl, 13 ; \r

int 21h

mov dl, 10 ; \n

int 21h

endm

check_bounds macro reg, l, h, err

cmp reg, l

j1 err

cmp reg, h

jg err

endm

input_str macro buffer

lea dx, buffer

mov ah, 0Ah

int 21h

endm

arr_sum macro arr, sum, overflow_err

local sum_loop

sum_loop:

mov ax, [arr]

add sum, ax

jo overflow_err ; if sum overflows

add arr, 2

loop sum_loop

endm

arr_max macro arr, reg

local max_loop

local skip_max

mov reg, [arr] ; first max - first element

```

max_loop:
    cmp    reg, [arr]
    jge    skip_max      ; max >= el - skip
    mov    reg, [arr]    ; otherwise update max
skip_max:
    add    arr, 2
    loop   max_loop
endm

arr_sort macro arr, arr_size
    local outer_sort
    local inner_sort
    local no_swap

outer_sort:
    push    cx
    mov     cx, arr_size
    dec     cx
    lea     si, arr
inner_sort:
    mov     ax, [si]
    mov     bx, [si+2]
    cmp     ax, bx
    jle     no_swap      ; first <= second - skip

    ; swap
    mov     [si], bx
    mov     [si+2], ax
no_swap:
    add     si, 2
    loop    inner_sort

    pop     cx
    loop    outer_sort
endm

print_arr macro arr
    local    print_arr_loop

print_arr_loop:
    mov     ax, [arr]
    push    cx
    push    arr
    call    OutputProc

    ; print space between numbers
    print_str msg_space

    pop     arr
    pop     cx

```

```

                                add      arr, 2
                                loop     print_arr_loop
endm

.data
; messages
msg_size      db 'Enter array size (1-20): $'
msg_elem      db 'Enter element: $'
msg_sum       db 13, 10, 'Sum of elements: $'
msg_max       db 13, 10, 'Maximum element: $'
msg_sort      db 13, 10, 'Sorted array: $'
msg_space     db ' $'

msg_err       db 13, 10, 'Error: Invalid input!$'
msg_ovf       db 13, 10, 'Error: Overflow! Limit to 16-bit.$'
msg_size_err  db 13, 10, 'Error: Size must be between 1 and 20.$'

; variables & array
buffer        db 7, 0, 7 dup(0)
arr           dw 20
dup(0)                                     ; reserved for 20
words (40 bytes)
arr_size      dw 0
is_neg        db 0

.code
main proc
                                mov      ax, @data
                                mov      ds, ax

                                ; input size
                                print_str msg_size

                                call      InputProc
                                jc        exit_prog

                                check_bounds ax, 1, 20, bad_size
                                mov      arr_size, ax

                                print_nl

                                ; input elements
                                mov      cx, arr_size
                                lea      di, arr

                                input_loop:
                                push     cx                                ; save
counter
                                print_str msg_elem

```

```

                                call      InputProc
                                pop       cx
                                jc        exit_prog          ; invalid
input
                                mov       [di], ax           ; in c => *di
= ax
                                add       di, 2              ; move 1 word
ahead

                                print_nl
                                loop     input_loop

                                ; find sum
                                print_str msg_sum
                                mov       cx, arr_size
                                lea       si, arr
                                xor       bx, bx             ; bx - sum

                                arr_sum  si, bx, overflow_err
                                mov       ax, bx             ; ax - 1st
argument

                                call      OutputProc

                                ; find max
                                print_str msg_max

                                mov       cx, arr_size
                                lea       si, arr

                                arr_max   si, ax
                                arr_max   si, ax
                                call      OutputProc

                                ; bubble sort
                                mov       cx, arr_size
                                dec       cx                 ; num of
iters: size - 1
                                cmp      cx, 0
                                je        print_sorted      ; only 1 el -
skip sort
                                arr_sort arr, arr_size

                                ; finally print ts
                                print_sorted:
                                print_str msg_sort

                                mov       cx, arr_size
                                lea       si, arr

```

```

        print_arr    si

        jmp          exit_prog

bad_size:
        print_str    msg_size_err
        jmp          exit_prog

overflow_err:
        print_str    msg_ovf
        jmp          exit_prog

exit_prog:
        mov          ax, 4c00h
        int          21h
main endp

```

```

; copied from lab2
; returns in ax

```

```

InputProc proc
        input_str    buffer

        lea          si, buffer+2
        mov          cl, buffer+1
        xor          ch, ch
        cmp          cx, 0
        je           input_error

        xor          bx, bx
        mov          is_neg, 0

        mov          al, [si]
        cmp          al, '-'
        jne          check_plus
        mov          is_neg, 1
        inc          si
        dec          cx
        jmp          parse_loop

check_plus:
        cmp          al, '+'
        jne          parse_loop
        inc          si
        dec          cx

parse_loop:
        cmp          cx, 0
        je           parse_done

```

```

        mov     al, [si]
        cmp     al, '0'
        jl      input_error
        cmp     al, '9'
        jg      input_error

        sub     al, '0'
        xor     ah, ah
        push    cx

        mov     cx, ax
        mov     ax, bx
        mov     dx, 10
        imul    dx
        jo      ovf_err_proc

        add     ax, cx
        jo      ovf_err_proc

        mov     bx, ax
        pop     cx

        inc     si
        dec     cx
        jmp     parse_loop

parse_done:
        cmp     is_neg, 1
        jne     save_number
        neg     bx
        jo      ovf_err_proc

save_number:
        mov     ax, bx
        cld
        ret

ovf_err_proc:
        pop     cx
        print_str msg_ovf
        stc
        ret

input_error:
        print_str msg_err
        stc
        ret

InputProc endp

```

```

OutputProc proc
    cmp     ax, 0
    jge     start_convert

    push    ax
    mov     ah, 02h
    mov     dl, '-'
    int     21h
    pop     ax
    neg     ax

start_convert:
    xor     cx, cx
    mov     bx, 10

divide_loop:
    xor     dx, dx
    div     bx
    push    dx
    inc     cx
    test    ax, ax
    jnz     divide_loop

print_loop:
    pop     dx
    add     dl, '0'
    mov     ah, 02h
    int     21h
    loop    print_loop

    ret

OutputProc endp

end main

```

Вміст .lst файлу 2

Turbo Assembler
1
LAB2.ASM

Version 4.0 05/08/26 30:02:24

Page

```

1      0000      .model small
2      0000      .stack 100h
3
4      print_str macro str
5          lea dx, str
6          mov ah, 09h
7          int 21h
8      endm

```

```

9
10      0000                                .data
11      0000  45 6E 74 65 72 20 61+  msg_input db 'Enter an
integer: $'
12          6E 20 69 6E 74 65 67+
13          65 72 3A 20 24
14      0013  0D 0A 52 65 73 75 6C+  msg_res    db 13, 10,
'Result (-34): $'
15          74 20 28 2D 33 34 29+
16          3A 20 24
17      0024  0D 0A 45 72 72 6F 72+  msg_err    db 13, 10,
'Error: Invalid input! (Numbers only)$'
18          3A 20 49 6E 76 61 6C+
19          69 64 20 69 6E 70 75+
20          74 21 20 28 4E 75 6D+
21          62 65 72 73 20 6F 6E+
22          6C 79 29 24
23      004B  0D 0A 45 72 72 6F 72+  msg_ovf    db 13, 10,
'Error: Overflow/Underflow! Keep it 16-bit.$'
24          3A 20 4F 76 65 72 66+
25          6C 6F 77 2F 55 6E 64+
26          65 72 66 6C 6F 77 21+
27          20 4B 65 65 70 20 69+
28          74 20 31 36 2D 62 69+
29          74 2E 24
30
31                                     ; Buf for 10th      DOS function: max
len, real len, char array
32      0078  07 00 07*(00)          buffer    db 7, 0, 7 dup(0)
33      0081  0000                      num      dw 0
34      0083  00                      is_neg     db 0
35
36      0084                                .code
37      0000                                main proc
38      0000  B8 0000s                      mov      ax, @data
39      0003  8E D8                      mov      ds, ax
40
41                                     print_str msg_input
1 42      0005  BA 0000r                      lea dx, msg_input
1 43      0008  B4 09                      mov ah, 09h
1 44      000A  CD 21                      int 21h
45
46      000C  E8 0028                      call     InputProc
47      000F  72 21                      jc      exit_prog ;
Carry Flag = 1 => error
48
49                                     ; Var. 19: -34
50      0011  A1 0081r                      mov      ax, num
51      0014  2D 0022                      sub      ax, 34
52      0017  70 0F                      jo      overflow_err

```

	53				
ax	54	0019	50	push	ax ; save
	55			print_str	msg_res
1	56	001A	BA 0013r	lea	dx, msg_res
1	57	001D	B4 09	mov	ah, 09h

2

LAB2.ASM

```
1      58      001F  CD  21                      int  21h
      59      0021  58                          pop      ax      ;
restore ax
      60
      61      0022  E8  0092                      call     OutputProc
      62      0025  EB  0B  90                      jmp      exit_prog
      63
      64      0028                      overflow_err:
      65                      print_str msg_ovf
1      66      0028  BA  004Br                      lea dx, msg_ovf
1      67      002B  B4  09                      mov ah, 09h
1      68      002D  CD  21                      int  21h
      69      002F  EB  01  90                      jmp      exit_prog
      70
      71      0032                      exit_prog:
      72      0032  B8  4C00                      mov      ax, 4c00h
      73      0035  CD  21                      int      21h
      74      0037                      main endp
      75
      76
      77
      78      0037                      InputProc proc
      79      0037  BA  0078r                      lea      dx, buffer
      80      003A  B4  0A                      mov      ah, 0Ah
      81      003C  CD  21                      int      21h
      82
      83      003E  BE  007Ar                      lea      si, buffer+2
      ; ptr of symbols
      84      0041  8A  0E  0079r                      mov      cl, buffer+1
      ; num of symbols
      85      0045  32  ED                      xor      ch, ch
      86      0047  83  F9  00                      cmp      cx, 0
      87      004A  74  62                      je input_error ;
zero characters
      88
      89      004C  33  DB                      xor      bx, bx
      ; num accumulator
      90      004E  C6  06  0083r 00                      mov      is_neg,
0
      91
      92                      ; check for sign
      93      0053  8A  04                      mov      al, [si]
      94      0055  3C  2D                      cmp      al, '-'
      95      0057  75  0A                      jne      check_plus
```

```

1      96      0059  C6 06 0083r 01                                mov      is_neg,
      97      005E  46                                inc      si
      98      005F  49                                dec      cx
      99      0060  EB 07 90                                jmp      parse_loop
100
101      0063                                check_plus:
102      0063  3C 2B                                cmp      al, '+'
103      0065  75 02                                jne      parse_loop
104      0067  46                                inc      si
105      0068  49                                dec      cx
106
107      0069                                parse_loop:
108      0069  83 F9 00                                cmp      cx, 0
109      006C  74 25                                je      parse_done
110
111      006E  8A 04                                mov      al, [si]
112                                ; check if it is a number
(0-9)
113      0070  3C 30                                cmp      al, '0'
114      0072  7C 3A                                jl      input_error

```

3

LAB2.ASM

```

115    0074  3C 39                cmp     al, '9'
116    0076  7F 36                jg     input_error
117
118    0078  2C 30                sub     al, '0'      ;
char -> num
119    007A  32 E4                xor     ah, ah
120    007C  51                  push    cx           ; save
counter
121
122    007D  8B C8                mov     cx, ax
      ; cx = current num
123    007F  8B C3                mov     ax, bx
124    0081  BA 000A              mov     dx, 10
125    0084  F7 EA                imul    dx           ;
ax = ax * 10
126    0086  70 1C                jo     ovf_error
127
128    0088  03 C1                add     ax, cx
      ; ax = ax + current num
129    008A  70 18                jo     ovf_error
130
131    008C  8B D8                mov     bx, ax
132    008E  59                  pop     cx           ;
restore counter
133
134    008F  46                  inc     si
135    0090  49                  dec     cx
136    0091  EB D6                jmp     parse_loop
137
138    0093                        parse_done:
139    0093  80 3E 0083r 01        cmp     is_neg,
1
140    0098  75 04                jne     save_number
141    009A  F7 DB                neg     bx           ;
swap left bit -> num is positive
142    009C  70 06                jo     ovf_error
143
144    009E                        save_number:
145    009E  89 1E 0081r          mov     num, bx
146    00A2  F8                  clc           ;
Carry Flag = 0
147    00A3  C3                  ret
148
149    00A4                        ovf_error:
```

```

150 00A4 59                                pop      cx                ;
clear_stack      when exiting loop
151                                print_str msg_ovf
1 152 00A5 BA 004Br                    lea dx, msg_ovf
1 153 00A8 B4 09                        mov ah, 09h
1 154 00AA CD 21                        int 21h
155 00AC F9                                stc                                ;
Carry Flag = 1
156 00AD C3                                ret
157
158 00AE                                input_error:
159                                print_str msg_err
1 160 00AE BA 0024r                    lea dx, msg_err
1 161 00B1 B4 09                        mov ah, 09h
1 162 00B3 CD 21                        int 21h
163 00B5 F9                                stc
164 00B6 C3                                ret
165 00B7                                InputProc endp
166
167
168
169 00B7                                OutputProc      proc
170 00B7 3D 0000                        cmp      ax, 0
171 00BA 7D 0A                        jge
start_convert      ; skip - if ax >= 0

```

4

LAB2.ASM

```
172
173                                ; print -
174    00BC    50                    push    ax
175    00BD    B4 02                mov     ah, 02h
176    00BF    B2 2D                mov     dl, '-'
177    00C1    CD 21                int     21h
178    00C3    58                    pop     ax
179    00C4    F7 D8                neg     ax            ;
swap left bit
180
181    00C6                                start_convert:
182    00C6    33 C9                xor     cx, cx
                                ; reset cx
183    00C8    BB 000A                mov     bx, 10
184    00CB                                divide_loop:
185    00CB    33 D2                xor     dx, dx
186    00CD    F7 F3                div     bx            ;
ax = ax / 10, dx = rem
187    00CF    52                    push    dx            ; push
to stack
188    00D0    41                    inc     cx
189    00D1    85 C0                test    ax, ax
190    00D3    75 F6                jnz     divide_loop
191
192    00D5                                print_loop:
193    00D5    5A                    pop     dx
194    00D6    80 C2 30                add     dl, '0'            ;
convert num    to char
195    00D9    B4 02                mov     ah, 02h
196    00DB    CD 21                int     21h
197    00DD    E2 F6                loop    print_loop
198
199    00DF    C3                    ret
200    00E0                                OutputProc    endp
201
202                                end main
```

Symbol Name	Type	Value
??DATE	Text	"05/08/26"
??FILENAME	Text	"LAB2"
??TIME	Text	"30:02:24"
??VERSION	Number	0400
@32BIT	Text	0
@CODE	Text	_TEXT
@CODESIZE	Text	0
@CPU	Text	0101H
@CURSEG	Text	_TEXT
@DATA	Text	DGROUP
@DATASIZE	Text	0
@FILENAME	Text	LAB2
@INTERFACE	Text	00H
@MODEL	Text	2
@STACK	Text	DGROUP
@WORDSIZE	Text	2
BUFFER	Byte	DGROUP:0078
CHECK_PLUS	Near	_TEXT:0063
DIVIDE_LOOP	Near	_TEXT:00CB
EXIT_PROG	Near	_TEXT:0032
INPUTPROC	Near	_TEXT:0037
INPUT_ERROR	Near	_TEXT:00AE
IS_NEG	Byte	DGROUP:0083
MAIN	Near	_TEXT:0000
MSG_ERR	Byte	DGROUP:0024
MSG_INPUT	Byte	DGROUP:0000
MSG_OVF	Byte	DGROUP:004B
MSG_RES	Byte	DGROUP:0013
NUM	Word	DGROUP:0081
OUTPUTPROC	Near	_TEXT:00B7
OVERFLOW_ERR	Near	_TEXT:0028
OVF_ERROR	Near	_TEXT:00A4
PARSE_DONE	Near	_TEXT:0093
PARSE_LOOP	Near	_TEXT:0069
PRINT_LOOP	Near	_TEXT:00D5
SAVE_NUMBER	Near	_TEXT:009E
START_CONVERT	Near	_TEXT:00C6

Macro Name

PRINT_STR

Groups & Segments

Bit Size Align Combine Class

DGROUP

Group

STACK	16	0100	Para	Stack	STACK
_DATA	16	0084	Word	Public	DATA
_TEXT	16	00E0	Word	Public	CODE

Вміст .lst файлу 3

Turbo Assembler

Version 4.0

05/23/26 28:09:58

Page

1
lab3.asm

```

1      0000      .model small
2              JUMPS
3      0000      .stack 100h
4
5      print_str macro str
6                  lea dx, str
7                  mov ah, 09h
8                  int 21h
9      endm
10
11     input_num macro var, str
12         print_str str
13         call      InputProc
14         jc         exit_prog      ; Exit
on error
15         mov        var, ax
16
17     endm
18
19     input_str macro buffer
20         lea dx, buffer
21         mov ah, 0Ah
22         int 21h
23     endm
24
25     0000      .data
26              ; Input messages
27     0000      45 6E 74 65 72 20 61+   msg_a          db      'Enter
a: '$'
28              3A 20 24
29     000A      0D 0A 45 6E 74 65 72+   msg_b          db      13, 10,
'Enter b: '$'
30              20 62 3A 20 24
31     0016      0D 0A 45 6E 74 65 72+   msg_x          db      13, 10,
'Enter x: '$'
32              20 78 3A 20 24
33
34              ; Output messages
35     0022      0D 0A 52 65 73 75 6C+   msg_res        db      13, 10,
'Result   Z = '$'
36              74 20 5A 20 3D 20 24
37     0030      20 28 72 65 6D 3A 20+   msg_rem        db      ' (rem:
'$'
38              24
39     0038      29 24                  msg_rem_end db      ')$'
40
```

```

41                                     ; Error messages
42    003A  0D 0A 45 72 72 6F 72+  msg_err      db      13, 10,
'Error:  Invalid   input! (Numbers      only)$'
43          3A 20 49 6E 76 61 6C+
44          69 64 20 69 6E 70 75+
45          74 21 20 28 4E 75 6D+
46          62 65 72 73 20 6F 6E+
47          6C 79 29 24
48    0061  0D 0A 45 72 72 6F 72+  msg_ovf      db      13, 10,
'Error:  Overflow/Underflow! Limit to 16-bit.$'
49          3A 20 4F 76 65 72 66+
50          6C 6F 77 2F 55 6E 64+
51          65 72 66 6C 6F 77 21+
52          20 4C 69 6D 69 74 20+
53          74 6F 20 31 36 2D 62+
54          69 74 2E 24
55
56                                     ; Variables
57    008F  07 00 07*(00)          buffer      db      7, 0, 7
dup(0)

```



```

100                                ; x > 0 => ax^2 + b/x
101    0050    calc_greater:
102                                ; Calculate ax^2
103    0050    A1 009Cr            mov     ax, val_x
104    0053    F7 E8              imul     ax                ;
ax = x^2 (dx:ax)
105    0055    71 03 E9 0094      jo      overflow_err
106    005A    F7 2E 0098r        imul     val_a
    ; ax = a * x^2
107    005E    71 03 E9 008B      jo      overflow_err
108    0063    8B D8              mov     bx, ax
    ; bx = ax^2
109
110                                ; Calculate b/x
111    0065    A1 009Ar            mov     ax, val_b
112    0068    99                  cwd
Extend ax to dx:ax for idiv
113    0069    F7 3E 009Cr        idiv     val_x
    ; ax, dx = rem
114

```

3

lab3.asm

```
115                                     ; Calculate (ax^2) + (b/x)
116    006D    03 C3                    add     ax, bx
117    006F    70 7D 90 90 90          jo     overflow_err
118
119                                     ; Save remaining
120    0074    89 16 009Fr              mov     res_rem, dx
121    0078    EB 47 90                jmp     print_res
122
123                                     ; x = 0 => a + 2b
124    007B                                calc_equal:
125    007B    A1 009Ar                  mov     ax, val_b
126    007E    B9 0002                  mov     cx, 2
127    0081    F7 E9                    imul    cx      ;
ax = 2b
128    0083    70 69 90 90 90          jo     overflow_err
129
130    0088    03 06 0098r              add     ax, val_a
131    ; ax = a + 2b
132    008C    70 60 90 90 90          jo     overflow_err
133    0091    EB 2E 90                jmp     print_res
134
135                                     ; x < 0 => ax^2 - bx
136    0094                                calc_less:
137                                     ; Calculate ax^2
138    0094    A1 009Cr                  mov     ax, val_x
139    0097    F7 E8                    imul    ax
140    0099    70 53 90 90 90          jo     overflow_err
141    009E    F7 2E 0098r              imul    val_a
142    00A2    70 4A 90 90 90          jo     overflow_err
143    00A7    8B D8                    mov     bx, ax
144    ; bx = ax^2
145
146                                     ; Calculate bx
147    00A9    A1 009Ar                  mov     ax, val_b
148    00AC    F7 2E 009Cr              imul    val_x
149    00B0    70 3C 90 90 90          jo     overflow_err
150    ; ax = bx
151
152                                     ; Calculate ax^2 - bx
153    00B5    2B D8                    sub     bx, ax
154    00B7    70 35 90 90 90          jo     overflow_err
155
156    00BC    8B C3                    mov     ax, bx
157    00BE    EB 01 90                jmp     print_res
158    00C1                                print_res:
```

```

159      00C1  50
result  Z
160
1  161      00C2  BA 0022r
1  162      00C5  B4 09
1  163      00C7  CD 21
      164      00C9  58
      165      00CA  E8 00CC
      ; Print first      part
      166
      167      00CD  83 3E 009Fr 00
res_rem, 0      ; Skip if 0
      168      00D2  74 24 90 90 90
      169
      170
      171

```

```

push     ax      ; Save

print_str msg_res
    lea dx, msg_res
    mov ah, 09h
    int 21h
pop      ax
    call OutputProc

    cmp

    je  exit_prog

; Print rem
print_str msg_rem

```

4

lab3.asm

```
1 172 00D7 BA 0030r      lea dx, msg_rem
1 173 00DA B4 09      mov ah, 09h
1 174 00DC CD 21      int 21h
175
176 00DE A1 009Fr      mov ax, res_rem
177 00E1 E8 00B5      call OutputProc
178
179
180 00E4 BA 0038r      print_str msg_rem_end
181 00E7 B4 09      lea dx, msg_rem_end
182 00E9 CD 21      mov ah, 09h
183 00EB EB 0B 90      int 21h
184
185
186 00EE      jmp exit_prog
187
188 00EE BA 0061r      overflow_err:
189 00F1 B4 09      print_str msg_ovf
190 00F3 CD 21      lea dx, msg_ovf
191 00F5 EB 01 90      mov ah, 09h
192
193 00F8      int 21h
194 00F8 B8 4C00      jmp exit_prog
195 00FB CD 21
196 00FD      exit_prog:
197
198
199
200      mov ax, 4c00h
201      int 21h
202
203      main endp
204
205
206
207      ; Copied from lab2
208      ; Returns in AX
209
210 InputProc proc
211 input_str buffer
212 00FD BA 008Fr      lea dx, buffer
213 0100 B4 0A      mov ah, 0Ah
214 0102 CD 21      int 21h
215
216 0104 BE 0091r      lea si, buffer+2
217 0107 8A 0E 0090r   mov cl, buffer+1
218 010B 32 ED      xor ch, ch
219 010D 83 F9 00      cmp cx, 0
220 0110 74 7E 90 90 90  je input_error
221
222 0115 33 DB      xor bx, bx
223 0117 C6 06 009Er 00  mov is_neg,
224
225 011C 8A 04      mov al, [si]
```

219	011E	3C 2D		cmp	al, '-'
220	0120	75 0D 90 90 90		jne	
check_plus					
221	0125	C6 06 009Er 01		mov	is_neg,
1					
222	012A	46		inc	si
223	012B	49		dec	cx
224	012C	EB 0A 90		jmp	parse_loop
225					
226	012F		check_plus:		
227	012F	3C 2B		cmp	al, '+'
228	0131	75 05 90 90 90		jne	
parse_loop					

```

229    0136  46                inc     si
230    0137  49                dec     cx
231
232    0138                                parse_loop:
233    0138  83 F9 00                cmp     cx, 0
234    013B  74 34 90 90 90          je     parse_done
235
236    0140  8A 04                mov     al, [si]
237    0142  3C 30                cmp     al, '0'
238    0144  7C 4A 90 90 90          jl     input_error
239    0149  3C 39                cmp     al, '9'
240    014B  7F 43 90 90 90          jg     input_error
241
242    0150  2C 30                sub     al, '0'
243    0152  32 E4                xor     ah, ah
244    0154  51                push    cx
245
246    0155  8B C8                mov     cx, ax
247    0157  8B C3                mov     ax, bx
248    0159  BA 000A                mov     dx, 10
249    015C  F7 EA                imul    dx
250    015E  70 26 90 90 90          jo     ovf_error
251
252    0163  03 C1                add     ax, cx
253    0165  70 1F 90 90 90          jo     ovf_error
254
255    016A  8B D8                mov     bx, ax
256    016C  59                pop     cx
257
258    016D  46                inc     si
259    016E  49                dec     cx
260    016F  EB C7                jmp     parse_loop
261
262    0171                                parse_done:
263    0171  80 3E 009Er 01          cmp     is_neg,
1
264    0176  75 0A 90 90 90          jne
save_number
265    017B  F7 DB                neg     bx
266    017D  70 07 90 90 90          jo     ovf_error
267
268    0182                                save_number:
269    0182  8B C3                mov     ax, bx
    ; Return in AX
270    0184  F8                cld
271    0185  C3                ret
272
273    0186                                ovf_error:

```

	274	0186	59		pop	cx
	275				print_str	msg_ovf
1	276	0187	BA 0061r		lea	dx, msg_ovf
1	277	018A	B4 09		mov	ah, 09h
1	278	018C	CD 21		int	21h
	279	018E	F9		stc	
	280	018F	C3		ret	
	281					
	282	0190		input_error:		
	283				print_str	msg_err
1	284	0190	BA 003Ar		lea	dx, msg_err
1	285	0193	B4 09		mov	ah, 09h

6

lab3.asm

```
1  286    0195    CD 21                      int 21h
    287    0197    F9                      stc
    288    0198    C3                      ret
    289    0199                      InputProc endp
    290
    291    0199                      OutputProc    proc
    292    0199    3D 0000                      cmp      ax, 0
    293    019C    7D 0D 90 90 90              jge
start_convert
    294
    295    01A1    50                      push     ax
    296    01A2    B4 02                      mov      ah, 02h
    297    01A4    B2 2D                      mov      dl, '-'
    298    01A6    CD 21                      int      21h
    299    01A8    58                      pop      ax
    300    01A9    F7 D8                      neg      ax
    301
    302    01AB                      start_convert:
    303    01AB    33 C9                      xor      cx, cx
    304    01AD    BB 000A                      mov      bx, 10
    305    01B0                      divide_loop:
    306    01B0    33 D2                      xor      dx, dx
    307    01B2    F7 F3                      div      bx
    308    01B4    52                      push     dx
    309    01B5    41                      inc      cx
    310    01B6    85 C0                      test     ax, ax
    311    01B8    75 F6                      jnz      divide_loop
    312
    313    01BA                      print_loop:
    314    01BA    5A                      pop      dx
    315    01BB    80 C2 30                      add      dl, '0'
    316    01BE    B4 02                      mov      ah, 02h
    317    01C0    CD 21                      int      21h
    318    01C2    E2 F6                      loop     print_loop
    319
    320    01C4    C3                      ret
    321    01C5                      OutputProc    endp
    322
    323                      end main
```

Symbol Name	Type	Value
??DATE	Text	"05/23/26"
??FILENAME	Text	"lab3"
??TIME	Text	"28:09:58"
??VERSION	Number	0400
@32BIT	Text	0
@CODE	Text	_TEXT
@CODESIZE	Text	0
@CPU	Text	0101H
@CURSEG	Text	_TEXT
@DATA	Text	DGROUP
@DATASIZE	Text	0
@FILENAME	Text	LAB3
@INTERFACE	Text	00H
@MODEL	Text	2
@STACK	Text	DGROUP
@WORDSIZE	Text	2
BUFFER	Byte	DGROUP:008F
CALC_EQUAL	Near	_TEXT:007B
CALC_GREATER	Near	_TEXT:0050
CALC_LESS	Near	_TEXT:0094
CHECK_PLUS	Near	_TEXT:012F
DIVIDE_LOOP	Near	_TEXT:01B0
EXIT_PROG	Near	_TEXT:00F8
INPUTPROC	Near	_TEXT:00FD
INPUT_ERROR	Near	_TEXT:0190
IS_NEG	Byte	DGROUP:009E
MAIN	Near	_TEXT:0000
MSG_A	Byte	DGROUP:0000
MSG_B	Byte	DGROUP:000A
MSG_ERR	Byte	DGROUP:003A
MSG_OVF	Byte	DGROUP:0061
MSG_REM	Byte	DGROUP:0030
MSG_REM_END	Byte	DGROUP:0038
MSG_RES	Byte	DGROUP:0022
MSG_X	Byte	DGROUP:0016
OUTPUTPROC	Near	_TEXT:0199
OVERFLOW_ERR	Near	_TEXT:00EE
OVF_ERROR	Near	_TEXT:0186
PARSE_DONE	Near	_TEXT:0171
PARSE_LOOP	Near	_TEXT:0138
PRINT_LOOP	Near	_TEXT:01BA
PRINT_RES	Near	_TEXT:00C1
RES_REM	Word	DGROUP:009F
SAVE_NUMBER	Near	_TEXT:0182
START_CONVERT	Near	_TEXT:01AB

VAL_A	Word	DGROUP:0098
VAL_B	Word	DGROUP:009A
VAL_X	Word	DGROUP:009C

Macro Name

INPUT_NUM
INPUT_STR
PRINT_STR

Groups & Segments

Bit Size Align Combine Class

DGROUP

Group

STACK
_DATA
_TEXT

16	0100	Para	Stack	STACK
16	00A1	Word	Public	DATA
16	01C5	Word	Public	CODE

Вміст .lst файлу 4

Turbo Assembler

Version 4.0

05/08/26 30:22:15

Page

1
lab4.asm

```
1      0000          .model small
2          JUMPS
3      0000          .stack 100h
4
5          print_str macro str
6              lea dx, str
7              mov ah, 09h
8              int 21h
9          endm
10
11         print_nl macro
12             mov ah, 02h
13             mov dl, 13      ; \r
14             int 21h
15             mov dl, 10      ; \n
16             int 21h
17         endm
18
19         check_bounds macro reg, l, h, err
20             cmp reg, l
21             jl  err
22             cmp reg, h
23             jg  err
24         endm
25
26         input_str macro buffer
27             lea dx, buffer
28             mov ah, 0Ah
29             int 21h
30         endm
31
32         arr_sum macro arr, sum, overflow_err
33             local sum_loop
34
35             sum_loop:
36                 mov     ax, [arr]
37                 add     sum, ax
38                 jo      overflow_err    ; if
sum overflows
39                 add     arr, 2
40                 loop    sum_loop
41         endm
42
43         arr_max macro arr, reg
44             local max_loop
45             local skip_max
```

```

46
47             mov     reg, [arr]      ; first
max - first element
48             max_loop:
49             cmp     reg, [arr]
50             jge     skip_max        ; max
>= el - skip
51             mov     reg, [arr]      ;
otherwise update max
52             skip_max:
53             add     arr, 2
54             loop    max_loop
55             endm
56
57             arr_sort macro arr, arr_size

```

```
58             local outer_sort
59             local inner_sort
60             local no_swap
61
62             outer_sort:
63                 push    cx
64                 mov     cx, arr_size
65                 dec     cx
66                 lea     si, arr
67             inner_sort:
68                 mov     ax, [si]
69                 mov     bx, [si+2]
70                 cmp     ax, bx
71                 jle     no_swap          ; first <=
second - skip
72
73                 ; swap
74                 mov     [si], bx
75                 mov     [si+2], ax
76             no_swap:
77                 add     si, 2
78                 loop    inner_sort
79
80                 pop     cx
81                 loop    outer_sort
82             endm
83
84             print_arr macro arr
85                 local      print_arr_loop
86
87             print_arr_loop:
88                 mov     ax, [arr]
89                 push    cx
90                 push    arr
91                 call    OutputProc
92
93                 ; print space between
numbers
94                 print_str msg_space
95
96                 pop     arr
97                 pop     cx
98                 add     arr, 2
99                 loop    print_arr_loop
100            endm
101
102            0000      .data
103                     ; messages
```

104	0000	45 6E 74 65 72 20	61+	msg_size	db 'Enter
		array size (1-20): \$'			
105		72 72 61 79 20 73	69+		
106		7A 65 20 28 31 2D	32+		
107		30 29 3A 20 24			
108	001A	45 6E 74 65 72 20	65+	msg_elem	db 'Enter
		element: \$'			
109		6C 65 6D 65 6E 74	3A+		
110		20 24			
111	002A	0D 0A 53 75 6D 20	6F+	msg_sum	db 13, 10,
		'Sum of elements: \$'			
112		66 20 65 6C 65 6D	65+		
113		6E 74 73 3A 20 24			
114	003E	0D 0A 4D 61 78 69	6D+	msg_max	db 13, 10,
		'Maximum element: \$'			

3

lab4.asm

```

115          75 6D 20 65 6C 65  6D+
116          65 6E 74 3A 20 24
117    0052  0D 0A 53 6F 72 74  65+  msg_sort      db 13, 10,
'Sorted array: $'
118          64 20 61 72 72 61  79+
119          3A 20 24
120    0063  20 24                      msg_space      db ' $'
121
122    0065  0D 0A 45 72 72 6F  72+  msg_err       db 13, 10,
'Error: Invalid input!$'
123          3A 20 49 6E 76 61  6C+
124          69 64 20 69 6E 70  75+
125          74 21 24
126    007D  0D 0A 45 72 72 6F  72+  msg_ovf       db 13, 10,
'Error: Overflow! Limit to 16-bit.$'
127          3A 20 4F 76 65 72  66+
128          6C 6F 77 21 20 4C  69+
129          6D 69 74 20 74 6F  20+
130          31 36 2D 62 69 74  2E+
131          24
132    00A1  0D 0A 45 72 72 6F  72+  msg_size_err  db 13, 10,
'Error: Size must be between 1 and 20.$'
133          3A 20 53 69 7A 65  20+
134          6D 75 73 74 20 62  65+
135          20 62 65 74 77 65  65+
136          6E 20 31 20 61 6E  64+
137          20 32 30 2E 24
138
139                                ; variables & array
140    00C9  07 00 07*(00)          buffer      db 7, 0, 7 dup(0)
141    00D2  14*(0000)              arr         dw 20 dup(0)
                                ; reserved      for 20 words+
                                (40 bytes)
142
143    00FA  0000                  arr_size     dw 0
144    00FC  00                   is_neg       db 0
145
146    00FD                                .code
147    0000                                main proc
148    0000  B8 0000s              mov         ax,
@data
149    0003  8E D8                mov         ds,  ax
150
151                                ; input size
152                                print_str    msg_size
1 153    0005  BA 0000r          lea dx, msg_size
1 154    0008  B4 09            mov ah, 09h
1 155    000A  CD 21            int 21h
156

```

```

157 000C E8 00FD
158 000F 73 03 E9 00F3
159
160
bad_size
1 161 0014 3D 0001
1 162 0017 7D 03 E9 00D7
1 163 001C 3D 0014
1 164 001F 7E 03 E9 00CF
165 0024 A3 00FAr
ax
166
167
1 168 0027 B4 02
1 169 0029 B2 0D
1 170 002B CD 21
1 171 002D B2 0A

```

```

call      InputProc
jc        exit_prog

check_bounds ax, 1, 20,

        cmp ax, 1
        jl bad_size
        cmp ax, 20
        jg bad_size
        mov          arr_size,

print_nl
mov ah, 02h
mov dl, 13      ; \r
int 21h
mov dl, 10      ; \n

```

4

lab4.asm

```
1 172 002F CD 21          int 21h
173
174          ; input elements
175 0031 8B 0E 00FAr      mov     cx,
    arr_size
176 0035 BF 00D2r        lea     di,  arr
177 0038                input_loop:
178 0038 51              push    cx
    ; save counter
179
180          print_str    msg_elem
1 181 0039 BA 001Ar      lea dx, msg_elem
1 182 003C B4 09        mov ah, 09h
1 183 003E CD 21        int 21h
184
185 0040 E8 00C9        call     InputProc
186 0043 59            pop     cx
187 0044 73 03 E9 00BE   jc     exit_prog
    ; invalid input
188
189 0049 89 05          mov     [di], ax
    ; in c => *di = ax
190 004B 83 C7 02        add     di,  2
    ; move 1 word ahead
191
192          print_nl
1 193 004E B4 02        mov ah, 02h
1 194 0050 B2 0D        mov dl, 13    ; \r
1 195 0052 CD 21        int 21h
1 196 0054 B2 0A        mov dl, 10    ; \n
1 197 0056 CD 21        int 21h
198 0058 E2 DE          loop
input_loop
199
200          ; find sum
201          print_str    msg_sum
1 202 005A BA 002Ar      lea dx, msg_sum
1 203 005D B4 09        mov ah, 09h
1 204 005F CD 21        int 21h
205 0061 8B 0E 00FAr      mov     cx,
    arr_size
206 0065 BE 00D2r        lea     si,  arr
207 0068 33 DB          xor     bx,  bx
    ; bx = sum
208
209          arr_sum     si,  bx,
overflow_err
1 210 006A            ??0000:
```

1	211	006A	8B 04		mov	ax, [si]	
1	212	006C	03 D8		add	bx, ax	
1	213	006E	71 03 E9 008A		jo	overflow_err	;
if bx overflows							
1	214	0073	83 C6 02		add	si, 2	
1	215	0076	E2 F2		loop	??0000	
	216	0078	8B C3		mov	ax, bx	
				; ax - 1st argument			
	217	007A	E8 012B		call		
OutputProc							
	218						
	219				; find max		
	220				print_str	msg_max	
1	221	007D	BA 003Er		lea dx,	msg_max	
1	222	0080	B4 09		mov ah,	09h	
1	223	0082	CD 21		int	21h	
	224						
	225	0084	8B 0E 00FAr		mov	cx,	
		arr_size					
	226	0088	BE 00D2r		lea	si,	arr
	227						
	228				arr_max	si,	ax

5

lab4.asm

```

1  229  008B  8B 04          mov     ax, [si]      ;
first max - first element
1  230  008D          ??0001:
1  231  008D  3B 04          cmp     ax, [si]
1  232  008F  7D 05 90 90 90      jge     ??0002
        ; max >= el - skip
1  233  0094  8B 04          mov     ax, [si]      ;
otherwise update max
1  234  0096          ??0002:
1  235  0096  83 C6 02          add     si, 2
1  236  0099  E2 F2          loop    ??0001
        237  009B  E8 010A      call
OutputProc
        238
        239                ; bubble sort
        240  009E  8B 0E 00FAr      mov     cx,
        arr_size
        241  00A2  49                dec     cx
        ; num of iters: size - 1
        242  00A3  83 F9 00          cmp     cx, 0
        243  00A6  74 25 90 90 90      je
print_sorted        ; only 1 el - skip sort
        244                arr_sort      arr,
arr_size
1  245  00AB          ??0003:
1  246  00AB  51                push    cx
1  247  00AC  8B 0E 00FAr      mov     cx, arr_size
1  248  00B0  49                dec     cx
1  249  00B1  BE 00D2r          lea     si, arr
1  250  00B4          ??0004:
1  251  00B4  8B 04          mov     ax, [si]
1  252  00B6  8B 5C 02          mov     bx, [si+2]
1  253  00B9  3B C3          cmp     ax, bx
1  254  00BB  7E 08 90 90 90      jle     ??0005      ;
first <= second - skip
1  255  00C0  89 1C          mov     [si], bx
1  256  00C2  89 44 02          mov     [si+2], ax
1  257  00C5          ??0005:
1  258  00C5  83 C6 02          add     si, 2
1  259  00C8  E2 EA          loop    ??0004
1  260  00CA  59                pop     cx
1  261  00CB  E2 DE          loop    ??0003
        262
        263
        264                ; finally print ts
        265  00CD          print_sorted:
        266                print_str      msg_sort
1  267  00CD  BA 0052r          lea     dx, msg_sort

```

1	268	00D0	B4 09		mov ah, 09h
1	269	00D2	CD 21		int 21h
	270				
	271	00D4	8B 0E 00FAr		mov cx,
		arr_size			
	272	00D8	BE 00D2r		lea si, arr
	273				print_arr si
1	274	00DB		??0006:	
1	275	00DB	8B 04		mov ax, [si]
1	276	00DD	51		push cx
1	277	00DE	56		push si
1	278	00DF	E8 00C6		call OutputProc
1	279				print_str msg_space
2	280	00E2	BA 0063r		lea dx, msg_space
2	281	00E5	B4 09		mov ah, 09h
2	282	00E7	CD 21		int 21h
1	283	00E9	5E		pop si
1	284	00EA	59		pop cx
1	285	00EB	83 C6 02		add si, 2

6

lab4.asm

```
1 286 00EE E2 EB          loop          ??0006
287
288 00F0 EB 15 90        jmp          exit_prog
289
290 00F3                  bad_size:
291                          print_str    msg_size_err
1 292 00F3 BA 00A1r      lea dx, msg_size_err
1 293 00F6 B4 09        mov ah, 09h
1 294 00F8 CD 21        int 21h
295 00FA EB 0B 90        jmp          exit_prog
296
297 00FD                  overflow_err:
298                          print_str    msg_ovf
1 299 00FD BA 007Dr      lea dx, msg_ovf
1 300 0100 B4 09        mov ah, 09h
1 301 0102 CD 21        int 21h
302 0104 EB 01 90        jmp          exit_prog
303
304 0107                  exit_prog:
305 0107 B8 4C00        mov          ax,
4c00h
306 010A CD 21          int          21h
307 010C                  main endp
308
309
310
311                      ; copied from lab2
312                      ; returns in ax
313
314 010C                  InputProc proc
315                          input_str    buffer
1 316 010C BA 00C9r      lea dx, buffer
1 317 010F B4 0A        mov ah, 0Ah
1 318 0111 CD 21        int 21h
319
320 0113 BE 00CBr        lea          si,
buffer+2
321 0116 8A 0E 00CAr    mov          cl,
buffer+1
322 011A 32 ED          xor          ch,    ch
323 011C 83 F9 00        cmp          cx,    0
324 011F 74 7E 90 90 90      je
input_error
325
326 0124 33 DB          xor          bx,    bx
327 0126 C6 06 00FCr 00      mov
is_neg, 0
328
```

```

329    012B  8A 04                                mov     al,
[si]
330    012D  3C 2D                                cmp     al, '-'
331    012F  75 0D 90 90 90                        jne
check_plus
332    0134  C6 06 00FCr 01                        mov
is_neg, 1
333    0139  46                                inc     si
334    013A  49                                dec     cx
335    013B  EB 0A 90                        jmp
parse_loop
336    013E                                check_plus:
337    013E  3C 2B                                cmp     al, '+'
338    0140  75 05 90 90 90                        jne
parse_loop
339    0145  46                                inc     si
340    0146  49                                dec     cx
341
342    0147                                parse_loop:

```

7

lab4.asm

```

    343    0147    83 F9 00                cmp     cx,    0
    344    014A    74 34 90 90 90          je
parse_done
    345
    346    014F    8A 04                mov     al,
    [si]
    347    0151    3C 30                cmp     al,    '0'
    348    0153    7C 4A 90 90 90          jl
input_error
    349    0158    3C 39                cmp     al,    '9'
    350    015A    7F 43 90 90 90          jg
input_error
    351
    352    015F    2C 30                sub     al,    '0'
    353    0161    32 E4                xor     ah,    ah
    354    0163    51                    push    cx
    355
    356    0164    8B C8                mov     cx,    ax
    357    0166    8B C3                mov     ax,    bx
    358    0168    BA 000A                mov     dx,    10
    359    016B    F7 EA                imul    dx
    360    016D    70 26 90 90 90          jo
ovf_err_proc
    361
    362    0172    03 C1                add     ax,    cx
    363    0174    70 1F 90 90 90          jo
ovf_err_proc
    364
    365    0179    8B D8                mov     bx,    ax
    366    017B    59                    pop     cx
    367
    368    017C    46                    inc     si
    369    017D    49                    dec     cx
    370    017E    EB C7                jmp
parse_loop
    371
    372    0180                                parse_done:
    373    0180    80 3E 00FCr 01          cmp
is_neg, 1
    374    0185    75 0A 90 90 90          jne
save_number
    375    018A    F7 DB                neg     bx
    376    018C    70 07 90 90 90          jo
ovf_err_proc
    377
    378    0191                                save_number:
    379    0191    8B C3                mov     ax,    bx
    380    0193    F8                    cld
```

	381	0194	C3		ret
	382				
	383	0195		ovf_err_proc:	
	384	0195	59	pop	cx
	385			print_str	msg_ovf
1	386	0196	BA 007Dr	lea dx,	msg_ovf
1	387	0199	B4 09	mov ah,	09h
1	388	019B	CD 21	int	21h
	389	019D	F9	stc	
	390	019E	C3	ret	
	391				
	392	019F		input_error:	
	393			print_str	msg_err
1	394	019F	BA 0065r	lea dx,	msg_err
1	395	01A2	B4 09	mov ah,	09h
1	396	01A4	CD 21	int	21h
	397	01A6	F9	stc	
	398	01A7	C3	ret	
	399	01A8		InputProc	endp

```

400
401      01A8      OutputProc      proc
402      01A8      3D 0000          cmp      ax,      0
403      01AB      7D 0D 90 90 90  jge
start_convert
404
405      01B0      50                push     ax
406      01B1      B4 02            mov      ah,      02h
407      01B3      B2 2D            mov      dl,      '-'
408      01B5      CD 21            int      21h
409      01B7      58                pop      ax
410      01B8      F7 D8            neg      ax
411
412      01BA      start_convert:
413      01BA      33 C9            xor      cx,      cx
414      01BC      BB 000A          mov      bx,      10
415      01BF      divide_loop:
416      01BF      33 D2            xor      dx,      dx
417      01C1      F7 F3            div      bx
418      01C3      52                push     dx
419      01C4      41                inc      cx
420      01C5      85 C0            test     ax,      ax
421      01C7      75 F6            jnz
divide_loop
422
423      01C9      print_loop:
424      01C9      5A                pop      dx
425      01CA      80 C2 30          add      dl,      '0'
426      01CD      B4 02            mov      ah,      02h
427      01CF      CD 21            int      21h
428      01D1      E2 F6            loop
print_loop
429
430      01D3      C3                ret
431      01D4      OutputProc      endp
432
433      end main

```

Symbol Name	Type	Value
??0000	Near	_TEXT:006A
??0001	Near	_TEXT:008D
??0002	Near	_TEXT:0096
??0003	Near	_TEXT:00AB
??0004	Near	_TEXT:00B4
??0005	Near	_TEXT:00C5
??0006	Near	_TEXT:00DB
??DATE	Text	"05/08/26"
??FILENAME	Text	"lab4"
??TIME	Text	"30:22:15"
??VERSION	Number	0400
@32BIT	Text	0
@CODE	Text	_TEXT
@CODESIZE	Text	0
@CPU	Text	0101H
@CURSEG	Text	_TEXT
@DATA	Text	DGROUP
@DATASIZE	Text	0
@FILENAME	Text	LAB4
@INTERFACE	Text	00H
@MODEL	Text	2
@STACK	Text	DGROUP
@WORDSIZE	Text	2
ARR	Word	DGROUP:00D2
ARR_SIZE	Word	DGROUP:00FA
BAD_SIZE	Near	_TEXT:00F3
BUFFER	Byte	DGROUP:00C9
CHECK_PLUS	Near	_TEXT:013E
DIVIDE_LOOP	Near	_TEXT:01BF
EXIT_PROG	Near	_TEXT:0107
INPUTPROC	Near	_TEXT:010C
INPUT_ERROR	Near	_TEXT:019F
INPUT_LOOP	Near	_TEXT:0038
IS_NEG	Byte	DGROUP:00FC
MAIN	Near	_TEXT:0000
MSG_ELEM	Byte	DGROUP:001A
MSG_ERR	Byte	DGROUP:0065
MSG_MAX	Byte	DGROUP:003E
MSG_OVF	Byte	DGROUP:007D
MSG_SIZE	Byte	DGROUP:0000
MSG_SIZE_ERR	Byte	DGROUP:00A1
MSG_SORT	Byte	DGROUP:0052
MSG_SPACE	Byte	DGROUP:0063
MSG_SUM	Byte	DGROUP:002A
OUTPUTPROC	Near	_TEXT:01A8

OVERFLOW_ERR	Near	_TEXT:00FD
OVF_ERR_PROC	Near	_TEXT:0195
PARSE_DONE	Near	_TEXT:0180
PARSE_LOOP	Near	_TEXT:0147
PRINT_LOOP	Near	_TEXT:01C9
PRINT_SORTED	Near	_TEXT:00CD
SAVE_NUMBER	Near	_TEXT:0191
START_CONVERT	Near	_TEXT:01BA

Macro Name

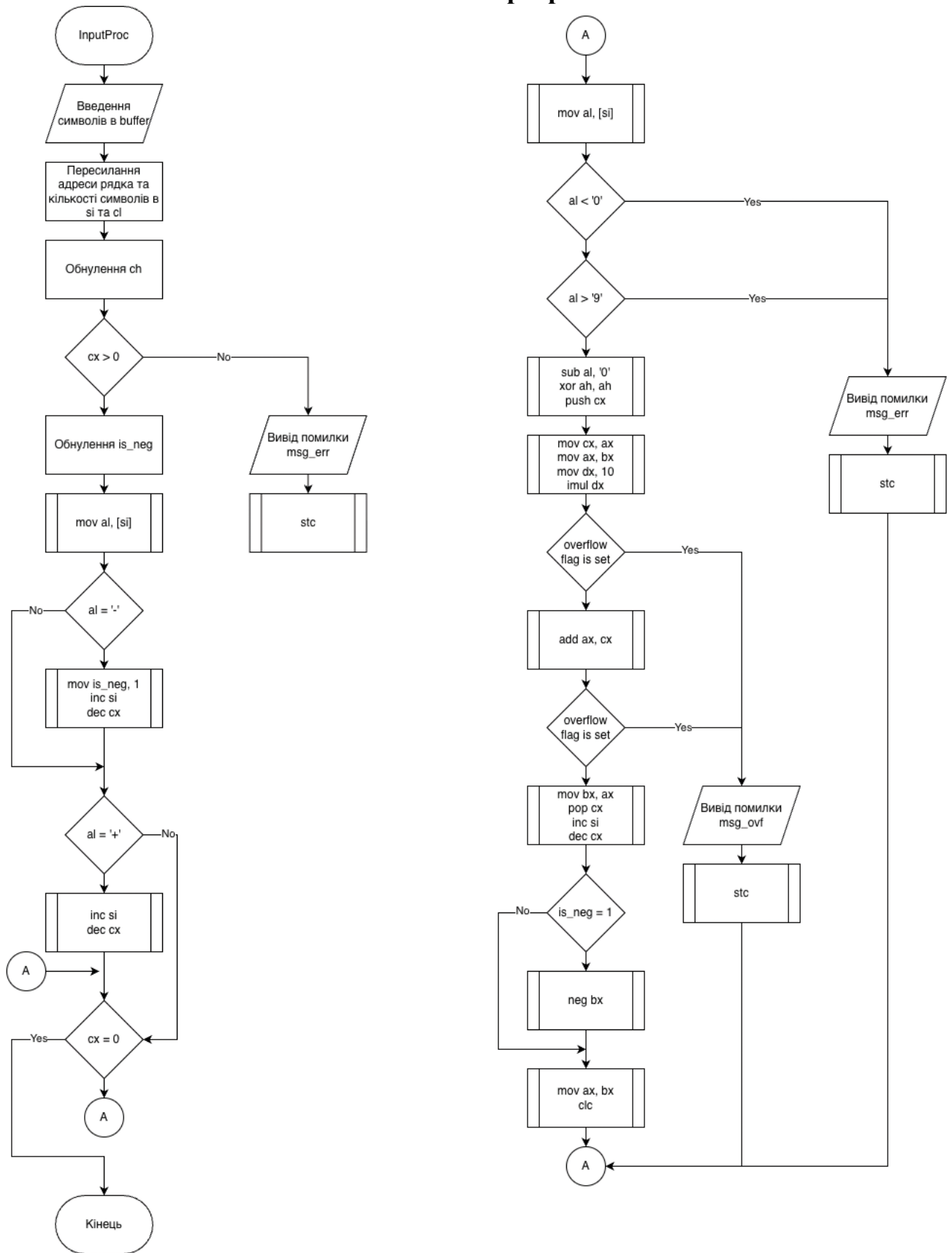
ARR_MAX
ARR_SORT
ARR_SUM
CHECK_BOUNDS
INPUT_STR
PRINT_ARR
PRINT_NL
PRINT_STR

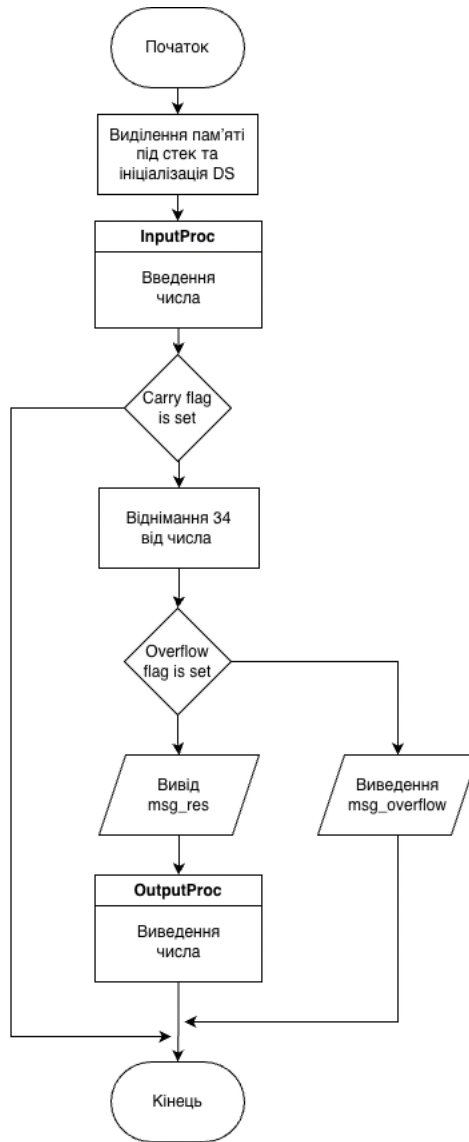
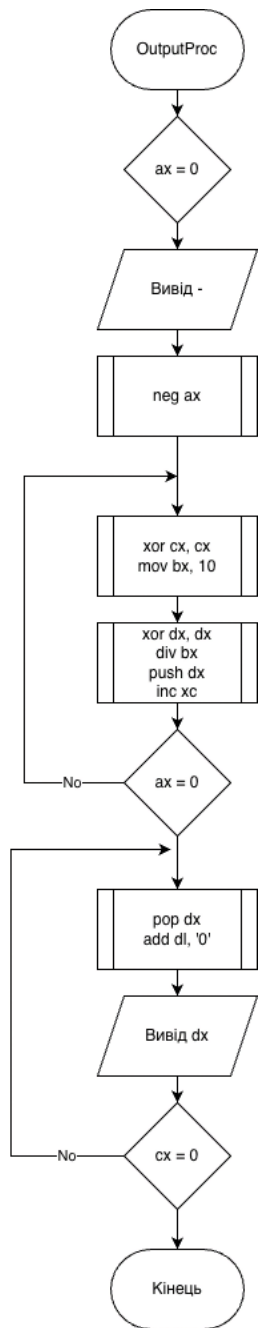
Groups & Segments

Bit Size Align Combine Class

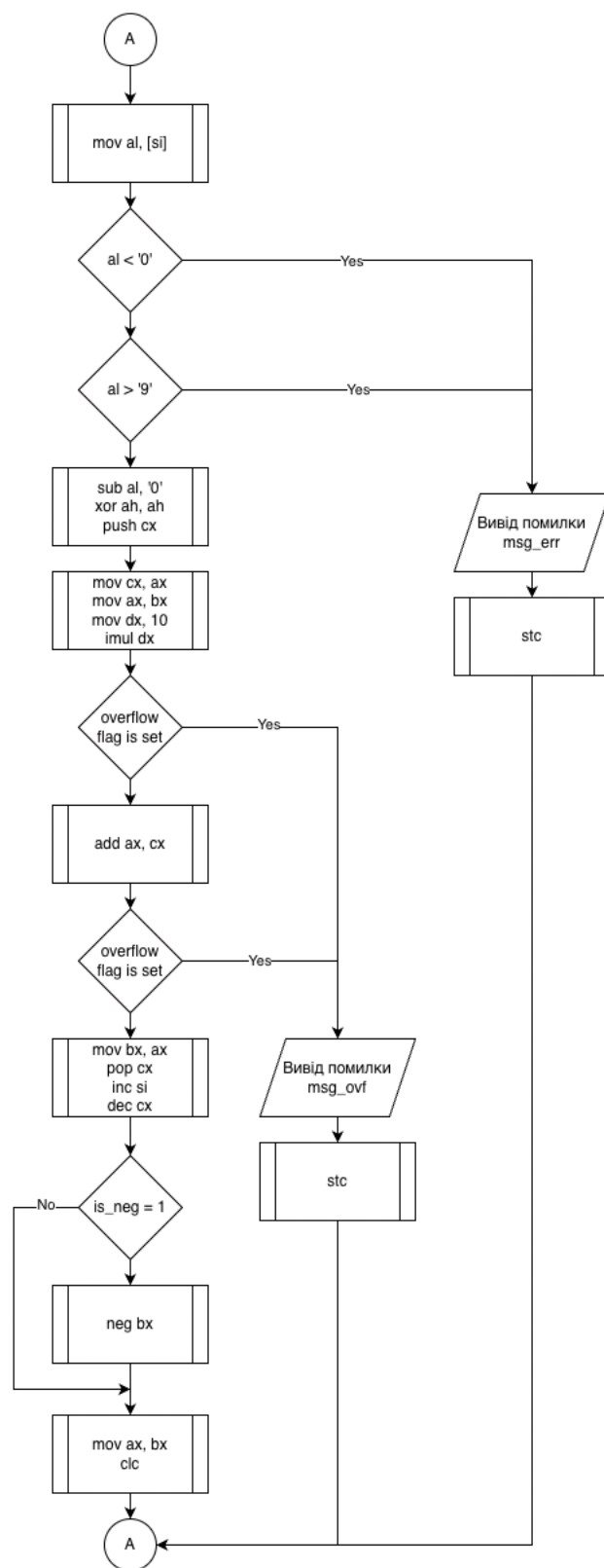
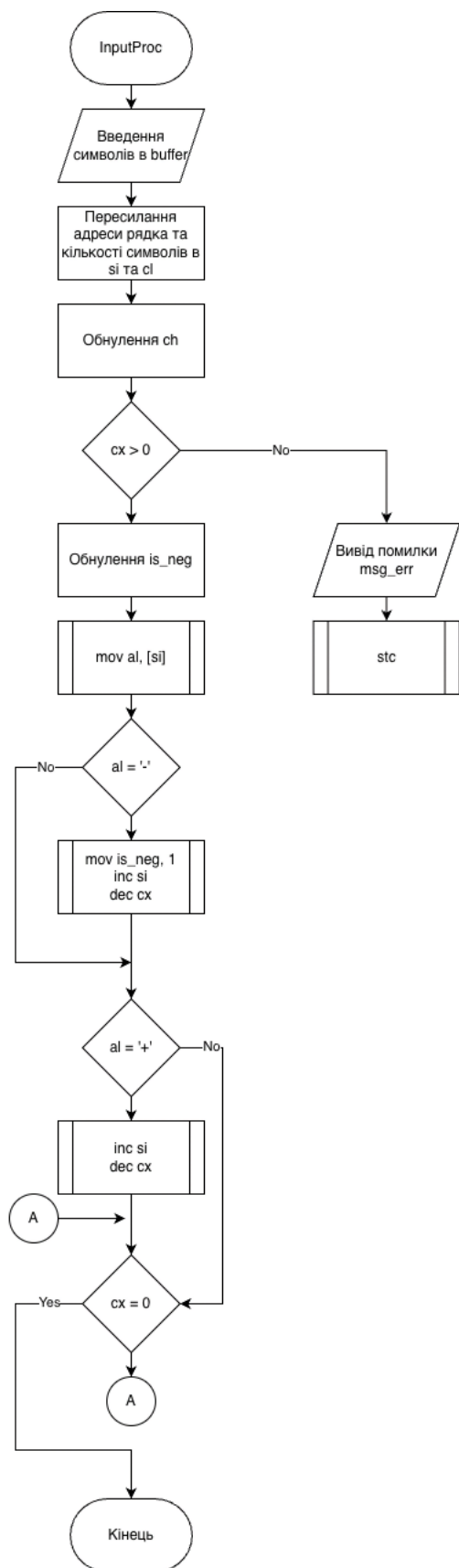
DGROUP	Group				
STACK	16	0100	Para	Stack	STACK
DATA	16	00FD	Word	Public	DATA
TEXT	16	01D4	Word	Public	CODE

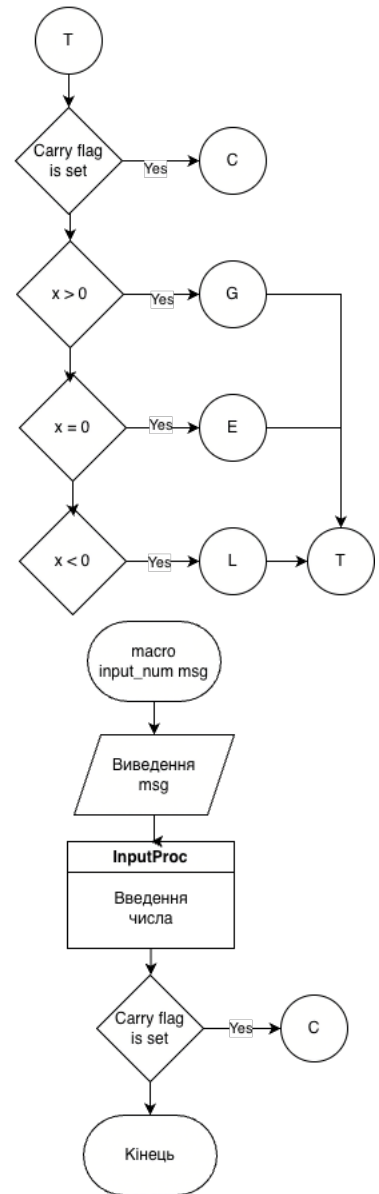
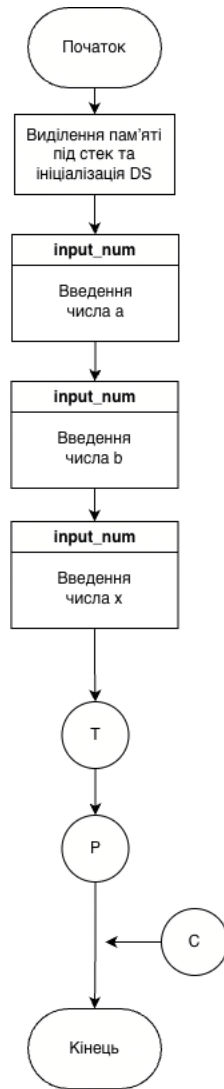
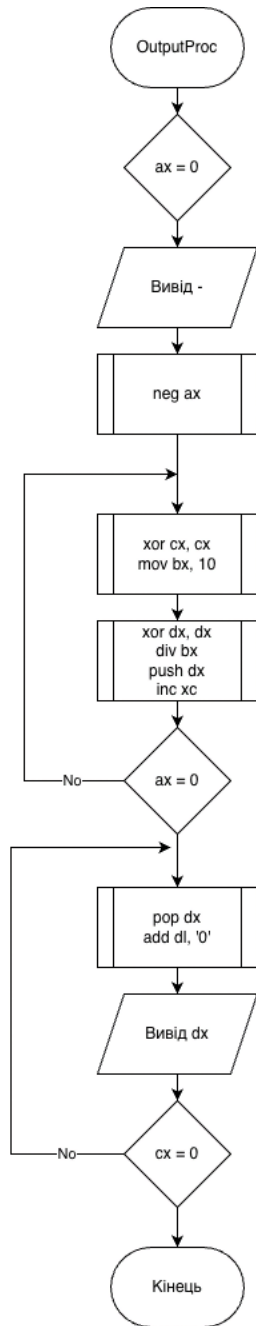
Блок схема програми 2

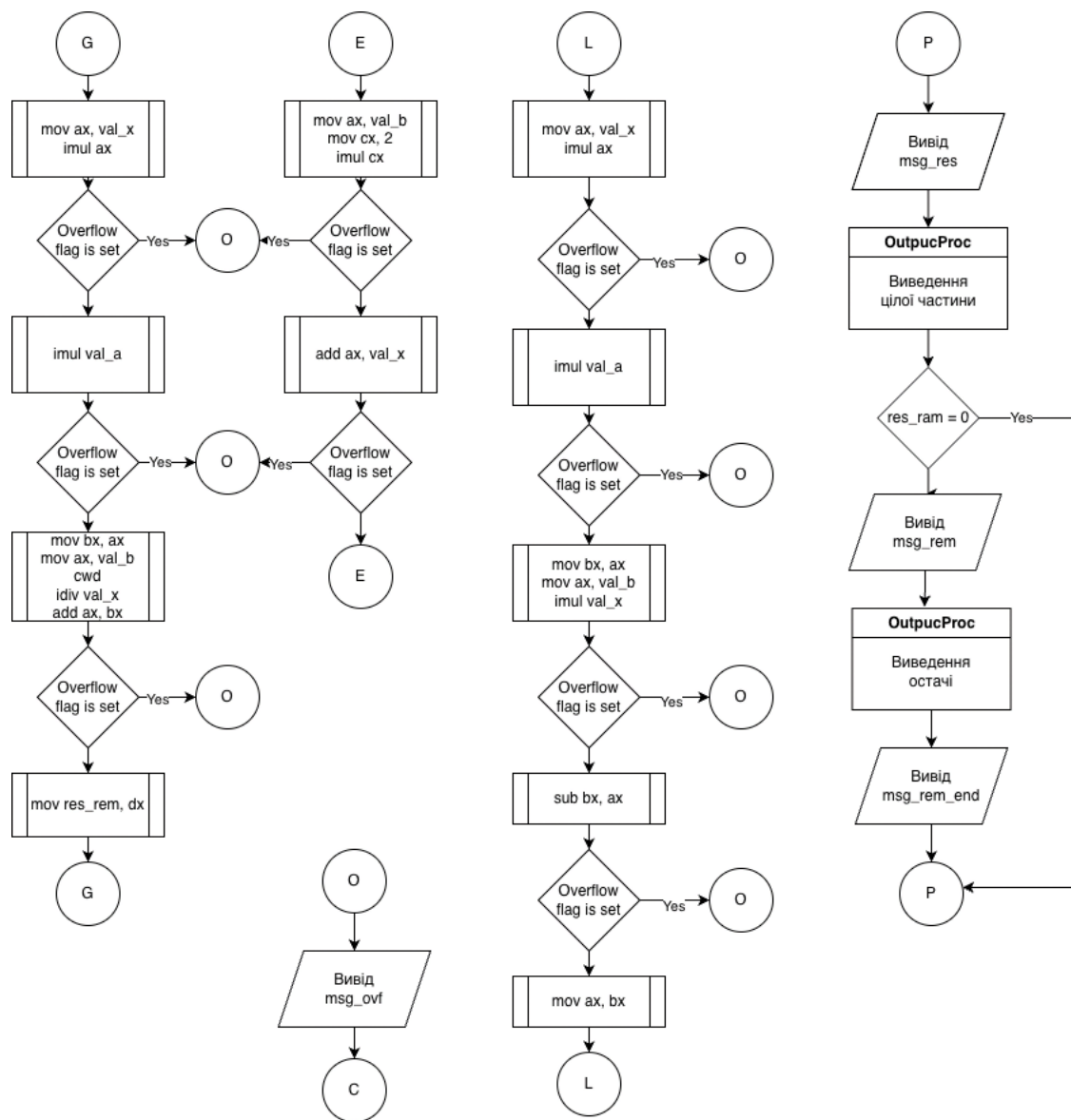




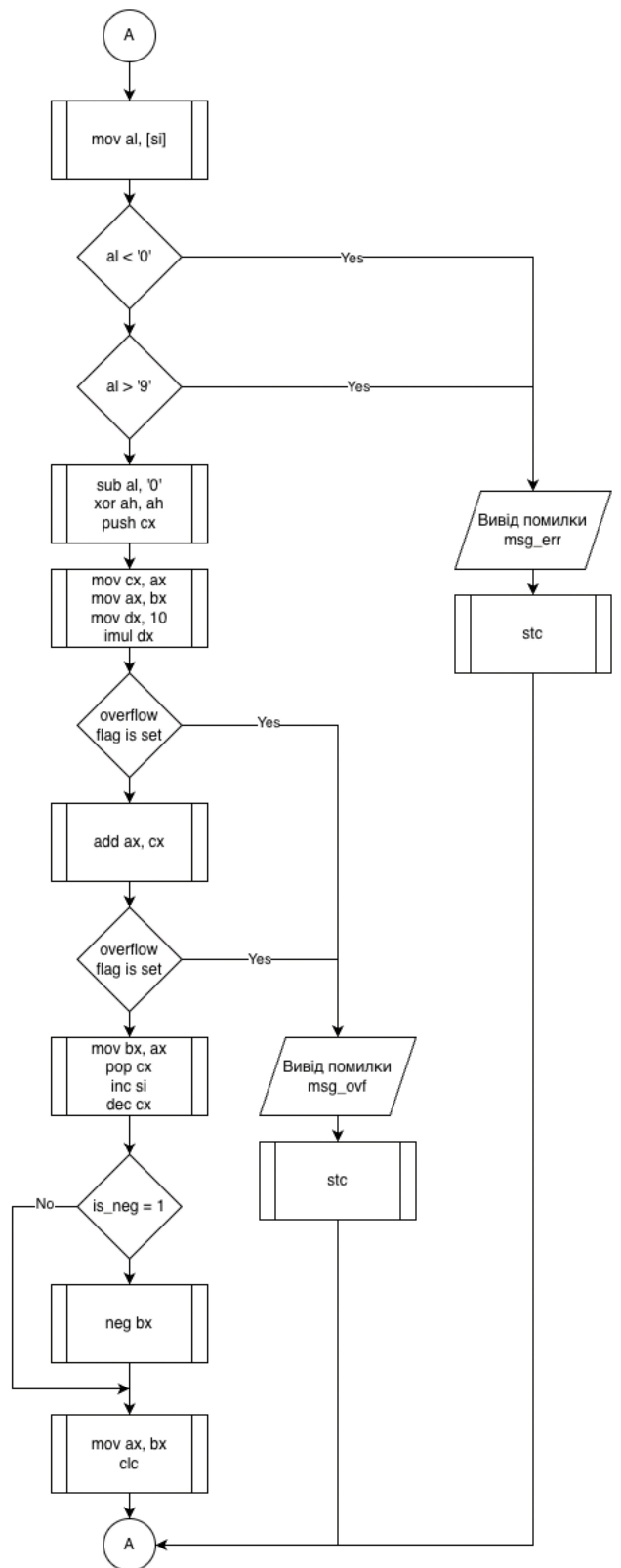
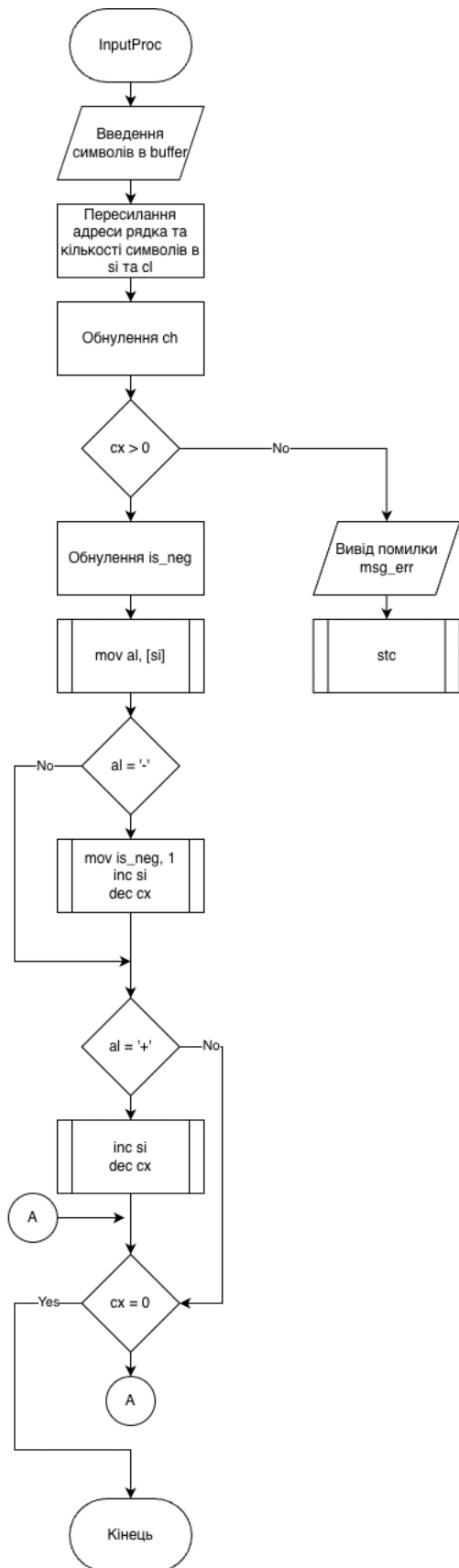
Блок схема програми 3

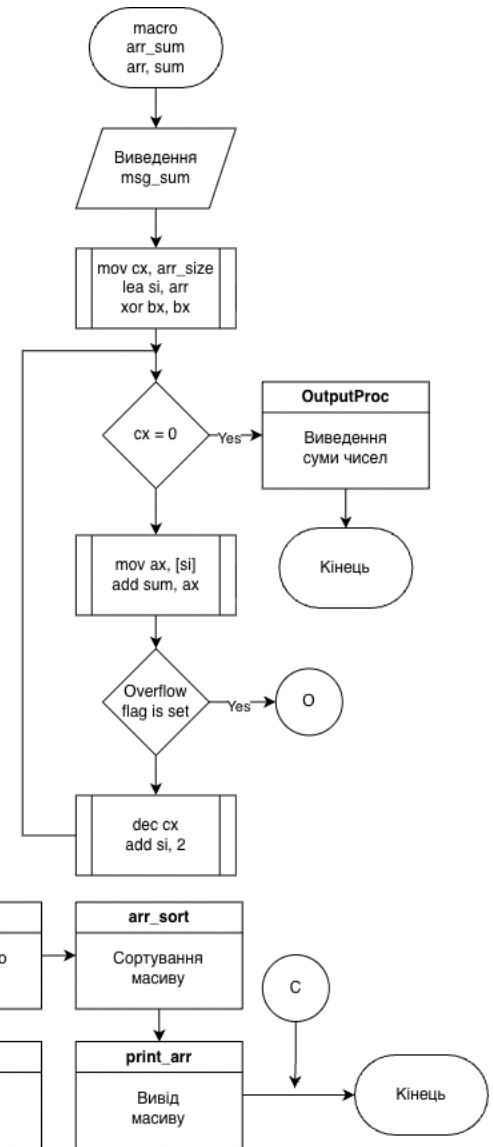
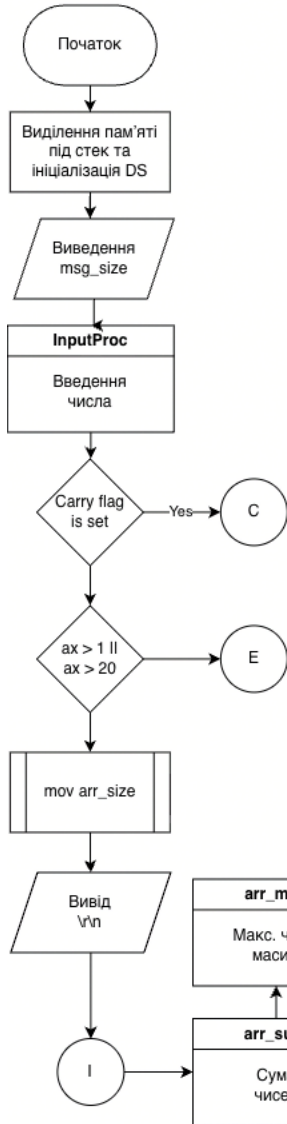
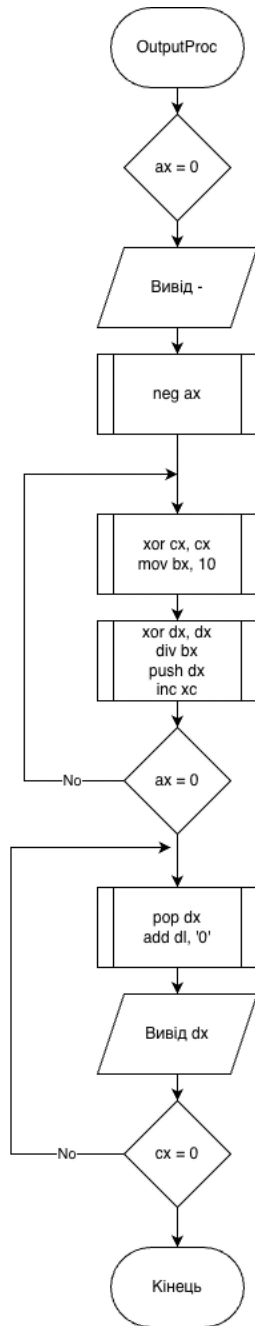


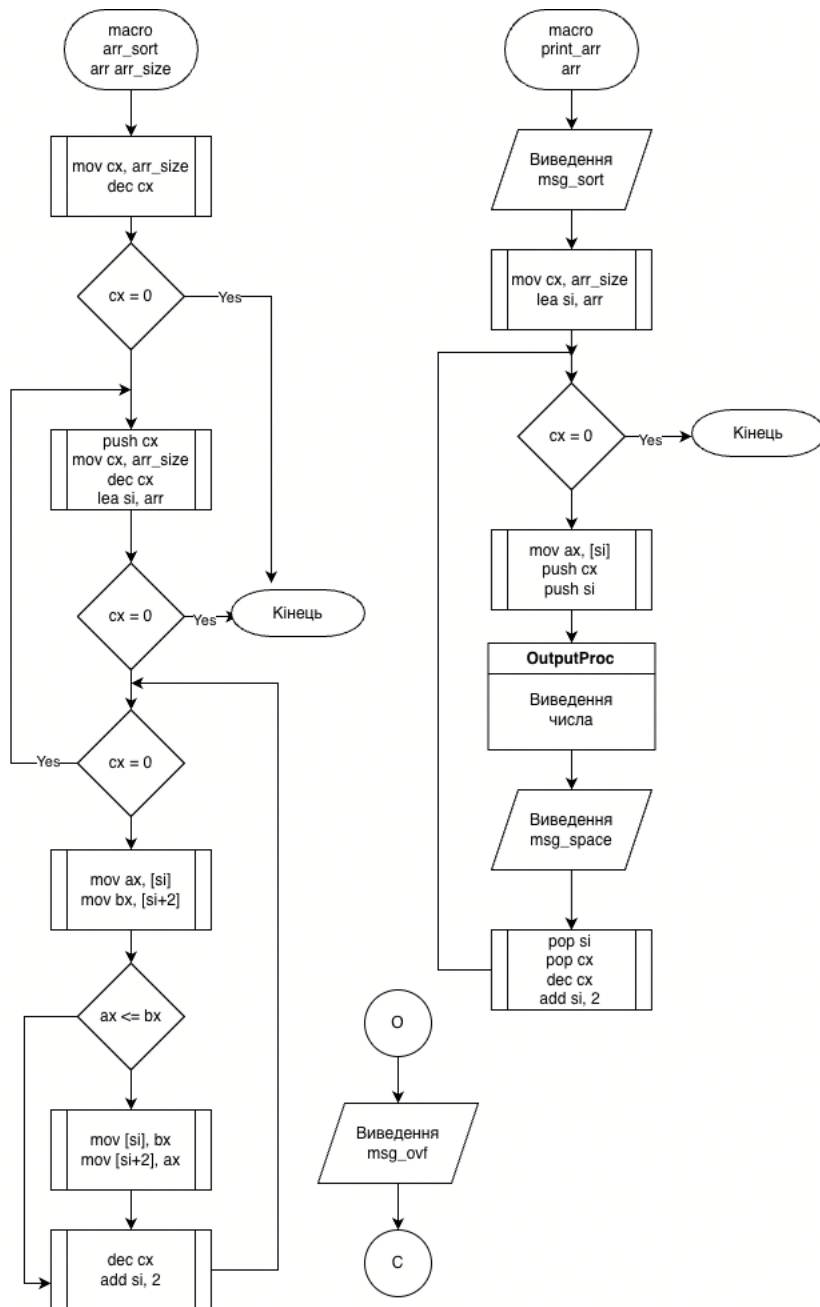


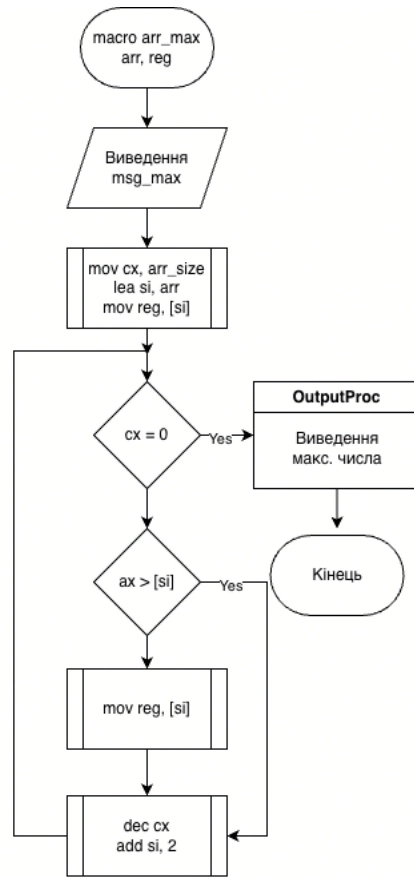
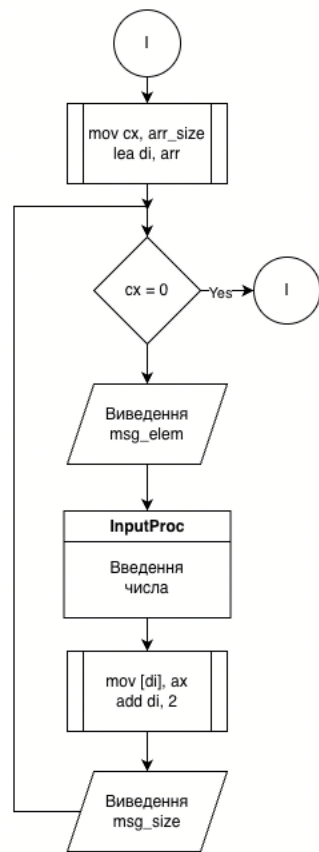


Блок схема програми 4









Приклад роботи програм

```
C:\LAB5>LAB2
Enter an integer: 1000
Result (-34): 966
C:\LAB5>LAB3
Enter a: 4
Enter b: 5
Enter x: 6
Result Z = 144 (rem: 5)
C:\LAB5>LAB4
Enter array size (1-20): 4
Enter element: 4
Enter element: 5
Enter element: 1
Enter element: 3

Sum of elements: 13
Maximum element: 5
Sorted array: 1 3 4 5
C:\LAB5>_
```

Висновок

Переписано КП 2, 3 та 4 з використанням макросів.